

Subqueries in PostgreSQL: Scalar, Row, Table & Correlated

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: What is a Subquery?](#)
 3. [Scalar Subqueries](#)
 4. [Row Subqueries](#)
 5. [Table Subqueries \(Derived Tables\)](#)
 6. [Correlated Subqueries](#)
 7. [Subqueries in WHERE](#)
 8. [Subqueries in FROM](#)
 9. [Subqueries in SELECT](#)
 10. [Subqueries in HAVING](#)
 11. [ASCII Visual Diagrams](#)
 12. [Subquery vs JOIN: When to Use Which](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Hands-on Exercises](#)
 18. [Advanced Notes](#)
 19. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Classify and write all types of subqueries
 - Use scalar, row, table, and correlated subqueries appropriately
 - Understand execution differences between subquery types
 - Rewrite correlated subqueries as JOINs for performance
 - Debug and optimize subquery-heavy queries
 - Explain subquery behavior in interviews
-

Theory: What is a Subquery?

A subquery (also called an inner query or nested query) is a SELECT statement embedded inside another SQL statement. The outer statement is called the main query or outer query.

Subquery Classification

By POSITION:

- In SELECT
- In FROM
- In WHERE

By RETURN TYPE:

- Scalar → single value (1 row, 1 col)
- Row → single row (1 row, N cols)
- Table → multiple rows and columns

- In HAVING
- Column → single column, multiple rows

By CORRELATION:

- Non-correlated (independent) → executes once
- Correlated (dependent) → executes once per outer row

Scalar Subqueries

A scalar subquery returns exactly one row and one column. It can be used anywhere a single value is expected.

ASCII Diagram

Main Query

```
SELECT name,  
       (SELECT MAX(salary)  
        FROM salaries) AS max_sal,  
       salary  
FROM employees
```

← Scalar Subquery
Returns ONE value

↑

Returns: one value inserted into every row of the outer query

Examples

```
-- Example 1: Scalar subquery in SELECT – compare each to overall average  
SELECT  
    rep_name,  
    amount,  
    (SELECT AVG(amount) FROM sales) AS overall_avg,  
    amount - (SELECT AVG(amount) FROM sales) AS diff_from_avg  
FROM sales  
ORDER BY diff_from_avg DESC;  
  
-- Example 2: Scalar subquery in WHERE – find above-average sales  
SELECT rep_name, amount, product  
FROM sales  
WHERE amount > (SELECT AVG(amount) FROM sales);  
  
-- Example 3: Scalar subquery with correlated table (preview of correlated)  
SELECT  
    r.region,  
    (SELECT SUM(amount) FROM sales s WHERE s.region = r.region) AS region_total  
FROM (SELECT DISTINCT region FROM sales WHERE region IS NOT NULL) r;  
  
-- Example 4: Scalar subquery in UPDATE  
UPDATE sales  
SET amount = amount * 1.10
```

```
WHERE amount < (SELECT AVG(amount) FROM sales);

-- Example 5: Scalar subquery returning NULL when no rows match
SELECT
    c.customer_name,
    (SELECT MAX(o.order_date) FROM orders o
     WHERE o.customer_id = c.customer_id) AS last_order_date
FROM customers c;
```

Row Subqueries

A row subquery returns exactly one row with multiple columns. Used with row constructors for multi-column comparisons.

Examples

```
-- Example 6: Row subquery with row constructor
SELECT rep_name, region, amount
FROM sales
WHERE (region, amount) = (
    SELECT region, MAX(amount)
    FROM sales
    WHERE region = 'North'
    GROUP BY region
);

-- Example 7: Row comparison with IN
SELECT employee_id, employee_name, department_id, salary
FROM employees
WHERE (department_id, salary) IN (
    SELECT department_id, MAX(salary)
    FROM employees
    GROUP BY department_id
);
```

Table Subqueries (Derived Tables)

A table subquery (derived table) appears in the FROM clause and returns a result set used as a virtual table.

ASCII Diagram

```
SELECT *
FROM (
    SELECT region,
           SUM(amount) AS rev
    FROM sales
    GROUP BY region
)
```

← Derived Table / Subquery
Executes FIRST
Returns a virtual table

```
) AS regional_summary          ← Must be aliased
WHERE rev > 3000;
```

Examples

```
-- Example 8: Derived table for a two-step aggregation
SELECT
    region,
    ROUND(AVG(rep_revenue), 2) AS avg_rep_revenue
FROM (
    SELECT region, rep_name, SUM(amount) AS rep_revenue
    FROM sales
    GROUP BY region, rep_name
) AS rep_totals
GROUP BY region
ORDER BY avg_rep_revenue DESC;

-- Example 9: Derived table to filter before joining
SELECT c.customer_name, top_orders.max_amount
FROM customers c
JOIN (
    SELECT customer_id, MAX(total_amount) AS max_amount
    FROM orders
    GROUP BY customer_id
) AS top_orders ON c.customer_id = top_orders.customer_id;

-- Example 10: Derived table with row_number for top-N per group
SELECT region, rep_name, amount
FROM (
    SELECT
        region,
        rep_name,
        amount,
        ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rn
    FROM sales
) AS ranked
WHERE rn = 1;

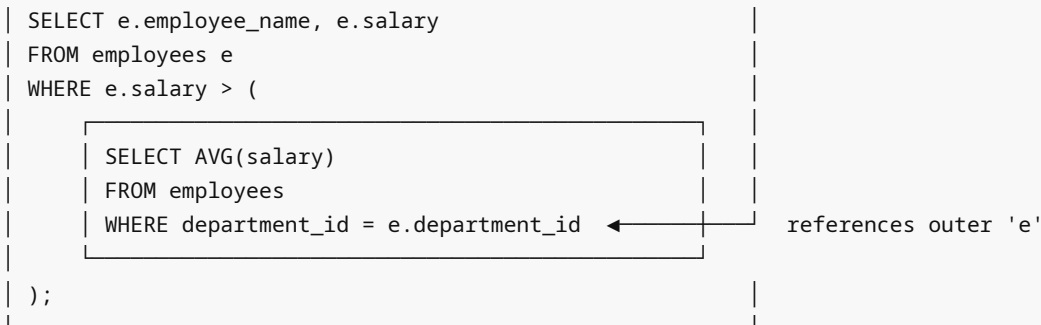
-- Example 11: Nested derived tables
SELECT outer_query.region, outer_query.avg_amount
FROM (
    SELECT region, AVG(amount) AS avg_amount
    FROM (
        SELECT region, amount
        FROM sales
        WHERE sale_date >= '2024-01-01'
    ) AS filtered
    GROUP BY region
) AS outer_query
WHERE avg_amount > 1500;
```

Correlated Subqueries

A correlated subquery references a column from the outer query. It is re-executed for each row processed by the outer query.

ASCII Diagram

Outer Query iterates row by row:



Execution: subquery runs ONCE PER ROW in outer query
If outer has 1000 rows → subquery executes 1000 times

Examples

-- Example 12: Correlated subquery – employees earning above dept average

```
SELECT  
  e.employee_name,  
  e.department_id,  
  e.salary,  
  (SELECT ROUND(AVG(salary),2) FROM employees  
   WHERE department_id = e.department_id) AS dept_avg  
FROM employees e  
WHERE e.salary > (  
  SELECT AVG(salary)  
  FROM employees  
  WHERE department_id = e.department_id  
);
```

-- Example 13: Correlated subquery – latest order per customer

```
SELECT  
  c.customer_name,  
  (SELECT order_date FROM orders o  
   WHERE o.customer_id = c.customer_id  
   ORDER BY order_date DESC  
   LIMIT 1) AS last_order_date  
FROM customers c;
```

-- Example 14: Correlated subquery to find the max-amount rep per region

```
SELECT rep_name, region, amount
```

```

FROM sales s1
WHERE s1.amount = (
    SELECT MAX(s2.amount)
    FROM sales s2
    WHERE s2.region = s1.region
);

-- Example 15: Correlated UPDATE – set each employee salary to their dept max
UPDATE employees e
SET salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE department_id = e.department_id
)
WHERE salary < (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);

-- Example 16: Correlated DELETE – remove duplicate rows, keep highest id
DELETE FROM sales s1
WHERE s1.sale_id < (
    SELECT MAX(s2.sale_id)
    FROM sales s2
    WHERE s2.rep_name = s1.rep_name
    AND s2.sale_date = s1.sale_date
    AND s2.product = s1.product
    AND s2.amount = s1.amount
);

```

Subqueries in WHERE

```

-- Example 17: IN with subquery – customers who placed orders
SELECT customer_name, city
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id FROM orders
    WHERE total_amount > 200
);

-- Example 18: NOT IN with subquery – customers with no orders
SELECT customer_name, city
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders
    WHERE customer_id IS NOT NULL -- IMPORTANT: always exclude NULLs
);

```

Subqueries in FROM

```
-- Example 19: Aggregated subquery in FROM – percentile buckets
SELECT
    bucket,
    COUNT(*) AS rep_count,
    SUM(total_rev) AS bucket_revenue
FROM (
    SELECT
        rep_name,
        SUM(amount) AS total_rev,
        NTILE(4) OVER (ORDER BY SUM(amount)) AS bucket
    FROM sales
    GROUP BY rep_name
) AS rep_buckets
GROUP BY bucket
ORDER BY bucket;
```

Subqueries in SELECT

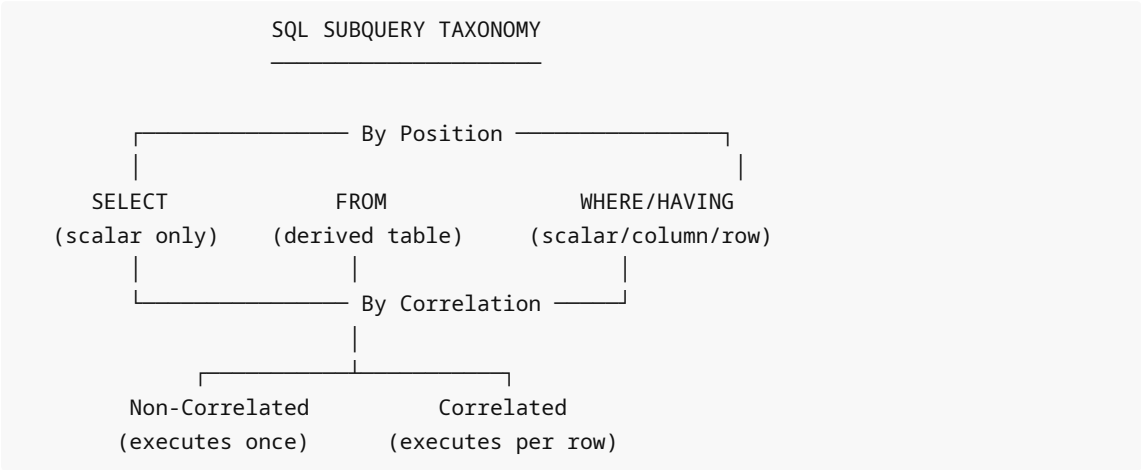
```
-- Example 20: Multiple scalar subqueries in SELECT
SELECT
    rep_name,
    amount,
    (SELECT MIN(amount) FROM sales) AS global_min,
    (SELECT MAX(amount) FROM sales) AS global_max,
    (SELECT AVG(amount) FROM sales) AS global_avg,
    ROUND(100.0 * amount / (SELECT SUM(amount) FROM sales), 2) AS pct_total
FROM sales
ORDER BY amount DESC;
```

Subqueries in HAVING

```
-- Example 21: HAVING with subquery – regions beating the overall average
SELECT
    region,
    SUM(amount) AS region_revenue
FROM sales
WHERE region IS NOT NULL
GROUP BY region
HAVING SUM(amount) > (SELECT AVG(region_total) FROM (
    SELECT SUM(amount) AS region_total
    FROM sales
    GROUP BY region
) AS rt);
```

ASCII Visual Diagrams

Subquery Taxonomy



Correlated Subquery Execution

Outer Table: 5 employees

emp_1	→	subquery runs	→	avg_dept_1 = 80000	→	compare	→	include/exclude
emp_2	→	subquery runs	→	avg_dept_1 = 80000	→	compare	→	include/exclude
emp_3	→	subquery runs	→	avg_dept_2 = 70000	→	compare	→	include/exclude
emp_4	→	subquery runs	→	avg_dept_2 = 70000	→	compare	→	include/exclude
emp_5	→	subquery runs	→	avg_dept_3 = 90000	→	compare	→	include/exclude

↑

5 executions of inner query

Subquery vs JOIN: When to Use Which

Use SUBQUERY when...	Use JOIN when...
You need a single value (scalar result)	You need columns from both tables in the output
You want to filter using an aggregated result	You want to combine and aggregate across tables
NOT IN / NOT EXISTS patterns	INNER JOIN for matched rows
The subquery is semantically clearer to the reader	JOIN is typically faster for large datasets

```
-- Subquery version: customers with orders over $200
SELECT customer_name FROM customers
```



```
WHERE customer_id IN (SELECT customer_id FROM orders WHERE total_amount > 200);

-- JOIN version (equivalent):
SELECT DISTINCT c.customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount > 200;
```

Common Mistakes

Mistake 1: Scalar subquery returning more than one row

```
-- WRONG: This crashes if multiple rows match
SELECT name FROM employees
WHERE salary = (SELECT salary FROM employees WHERE department_id = 1);
-- ERROR: more than one row returned by a subquery

-- CORRECT: Use MAX/MIN or LIMIT 1
WHERE salary = (SELECT MAX(salary) FROM employees WHERE department_id = 1);
```

Mistake 2: NOT IN with NULLs — the NULL trap

```
-- WRONG: NOT IN with NULLs returns no rows
SELECT customer_name FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
-- If ANY customer_id in orders is NULL, the whole result is empty!

-- CORRECT: Filter NULLs explicitly OR use NOT EXISTS
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT NULL);
-- OR
WHERE NOT EXISTS (SELECT 1 FROM orders WHERE orders.customer_id = customers.customer_id);
```

Mistake 3: Forgetting to alias derived tables

```
-- WRONG:
SELECT * FROM (SELECT id, name FROM employees);

-- CORRECT:
SELECT * FROM (SELECT id, name FROM employees) AS emp_list;
```

Mistake 4: Correlated subquery in SELECT causing N+1 performance

```
-- SLOW: Runs once per customer row
SELECT c.customer_name,
```

```

        (SELECT COUNT(*) FROM orders WHERE customer_id = c.customer_id) AS cnt
FROM customers c;

-- FAST: Use JOIN with aggregation
SELECT c.customer_name, COUNT(o.order_id) AS cnt
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;

```

Best Practices

1. Use CTEs (WITH clause) instead of deeply nested subqueries for readability.
2. Prefer NOT EXISTS over NOT IN when the subquery column can have NULLs.
3. Move non-correlated scalar subqueries to JOIN for large datasets.
4. Always alias derived tables in FROM clause.
5. Use LIMIT 1 on correlated subqueries when you expect at most one row.
6. Validate that scalar subqueries never return more than one row.
7. Prefer EXISTS over IN for semi-join patterns — the planner can short-circuit.

Performance Considerations

```

-- 1. PostgreSQL often converts subqueries to JOINS internally (unnesting)
-- Check with EXPLAIN to see if your subquery was rewritten:
EXPLAIN SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM
orders);

-- 2. Correlated subqueries that cannot be unnested run once per outer row
-- Replace with a JOIN + aggregation when possible

-- 3. Use CTE to compute once and reuse (pre-materialization in older PG versions)
-- In PostgreSQL 12+, CTEs are inlined by default (not materialized barriers)
-- Force materialization when needed for performance:
WITH expensive AS MATERIALIZED (
    SELECT region, SUM(amount) AS rev FROM sales GROUP BY region
)
SELECT * FROM expensive WHERE rev > 2000;

-- 4. Indexes on the subquery's WHERE predicate column are critical
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
-- This makes the correlated subquery: (SELECT COUNT(*) FROM orders WHERE
customer_id = ?)
-- use an index instead of a seq scan

```

Interview Questions & Answers

Q1: What are the types of subqueries?

A: By return type: scalar (1 row, 1 col), row (1 row, N cols), table (N rows, N cols), column (N rows, 1 col). By correlation: non-correlated (executes once) and correlated (executes per outer row). By position: in SELECT, FROM, WHERE, or HAVING.

Q2: What is a correlated subquery and what is its performance implication?

A: A correlated subquery references a column from the outer query, causing it to re-execute for every row in the outer query. For an outer table with N rows, the subquery runs N times. This is the "N+1 query problem" and can be catastrophic for large tables. Solution: rewrite as a JOIN with aggregation.

Q3: Why does NOT IN behave unexpectedly with NULL values?

A: SQL uses three-valued logic (TRUE, FALSE, UNKNOWN). When comparing a value with NULL using =, the result is UNKNOWN. NOT IN internally uses <> comparisons; if ANY value in the subquery is NULL, every comparison becomes UNKNOWN, and NOT IN returns no rows. Use NOT EXISTS or filter NULLs from the subquery.

Q4: What is a derived table?

A: A derived table is a subquery in the FROM clause, aliased as a virtual table. It is evaluated first and its result is used by the outer query. Unlike CTEs, derived tables are not named for reuse.

Q5: When would you use a subquery over a CTE?

A: Use a subquery when it is used in only one place and is simple enough to read inline. Use a CTE when the same result is referenced multiple times, or when you want to break a complex query into named, readable steps.

Q6: How does PostgreSQL handle subquery optimization?

A: The planner attempts to "unnest" (inline) subqueries and convert them to equivalent JOIN operations. For IN/EXISTS subqueries, it may use semi-join or anti-semi-join plan nodes. Correlated subqueries that cannot be unnested remain as "InitPlan" or "SubPlan" nodes in the plan.

Q7: How do you find the second-highest salary using a subquery?

A:

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Or using DISTINCT with OFFSET:

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Q8: What is the difference between IN and EXISTS for semi-joins?

A: IN materializes the subquery result and checks membership. EXISTS uses short-circuit evaluation — it stops as soon as the first match is found. EXISTS is generally faster when the subquery would return many rows, and handles NULLs correctly. IN can be faster when the subquery returns few rows and has a good index.

Hands-on Exercises

Exercise 1: Above-Average Sales

Find all sales where the amount is above the overall average amount. Show rep name, amount, and the average.

```
-- Solution:
SELECT
    rep_name,
    amount,
    ROUND((SELECT AVG(amount) FROM sales), 2) AS overall_avg
FROM sales
WHERE amount > (SELECT AVG(amount) FROM sales)
ORDER BY amount DESC;
```

Exercise 2: Top Earner per Region

Find the rep with the highest total revenue in each region using a subquery.

```
-- Solution:
SELECT rep_name, region, total_rev
FROM (
    SELECT
        rep_name,
        region,
        SUM(amount) AS total_rev,
        RANK() OVER (PARTITION BY region ORDER BY SUM(amount) DESC) AS rnk
    FROM sales
    WHERE region IS NOT NULL
    GROUP BY rep_name, region
) AS ranked
WHERE rnk = 1
ORDER BY region;
```

Exercise 3: Customers Without Recent Orders

Find customers who have not placed any order in 2024.

```
-- Solution using NOT EXISTS:
SELECT customer_name, city
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
        AND o.order_date >= '2024-01-01'
);
```

Exercise 4: Two-Level Aggregation

Find the average of each region's total revenue (i.e., average-of-sums).

```
-- Solution using derived table:
SELECT ROUND(AVG(region_total), 2) AS avg_region_revenue
```

```
FROM (  
    SELECT region, SUM(amount) AS region_total  
    FROM sales  
    WHERE region IS NOT NULL  
    GROUP BY region  
) AS region_sums;
```

Exercise 5: Duplicate Detection

Find all reps who sold the same product on the same date more than once (possible duplicate entries).

```
-- Solution:  
SELECT rep_name, product, sale_date, COUNT(*) AS count  
FROM sales  
GROUP BY rep_name, product, sale_date  
HAVING COUNT(*) > 1;
```

Advanced Notes

Lateral Subqueries

A LATERAL subquery in FROM can reference columns from preceding FROM items (a correlated derived table):

```
SELECT c.customer_name, recent.order_date, recent.total_amount  
FROM customers c  
LEFT JOIN LATERAL (  
    SELECT order_date, total_amount  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
    ORDER BY order_date DESC  
    LIMIT 1  
) AS recent ON true;
```

Subquery Factoring with CTE

For complex multi-step transformations, CTEs outperform nested subqueries in readability:

```
WITH  
regional AS (  
    SELECT region, SUM(amount) AS rev FROM sales GROUP BY region  
) ,  
ranked AS (  
    SELECT *, RANK() OVER (ORDER BY rev DESC) AS rnk FROM regional  
)  
SELECT * FROM ranked WHERE rnk <= 3;
```

Cross-references

- [04_exists_any_all.md](#) — EXISTS, ANY, ALL operators built on subquery patterns

- **04_ctes.md** (04_Advanced_SQL) — CTEs as readable subquery alternatives
- **01_joins_complete.md** — JOIN as an alternative to subqueries
- **02_aggregations.md** — Aggregations inside subqueries